

POCKETSMART AI

YOUR SMART BUDGET & RECOMMENDATION ASSISTANT

2nd - BSc Computer Science

TEAM LEADER

Yasvini R S

yasvini12senthil08@gmail.com

TEAM MEMBERS

1. Pavithra Kamaraj

202500250@sigc.edu

2. Paulin Seema Antony

202500170@sigc.edu

3. Narmathaa S

Narmathaasakthivelu8@gmail.com

PocketSmart AI

AI-Powered Smart Planning Web Application

1. Introduction

PocketSmart AI is a lightweight, browser-based AI smart planner that brings together several everyday planning tasks — home budgeting, jewelry purchasing plans, party organization, and tailored recommendations — into a single web page. The application is built on a FastAPI backend and a simple HTML/CSS frontend rendered through Jinja2 templates, drawing its intelligence from the Google Gemini API.

Rather than offering a separate form for every feature, PocketSmart AI presents one unified dashboard screen. The user picks the task they want from a “Select Task” dropdown, types their request or budget details into a single text box, and clicks Generate. The request is sent to the FastAPI backend, processed by the relevant Python module, passed on to the Gemini model, and the AI-generated response is rendered back on the same page under a “Result” heading — without a page reload.

The goal of the project is to give users a fast, low-friction planning companion that runs locally with minimal setup, while still being powered by a capable large language model in the cloud.

2. Problem Statement

Individuals trying to plan budgets, jewelry purchases, or social events typically have to juggle several disconnected tools: spreadsheets for calculations, separate apps for event planning, search engines for recommendations, and unstructured notes for tracking expenses. Switching between these tools causes confusion and slows down decision-making.

There is a need for a single, lightweight application that a user can open locally, type their planning goals or financial details into, and immediately get a useful, well-structured response — whether that response is a detailed home budget breakdown, a jewelry purchase schedule, a party cost estimator, or a smart recommendation.

3. Objectives

- Provide instant, AI-generated financial and event planning insights through a simple text prompt.
- Break down complex budgeting goals into clear, actionable, and beginner-friendly steps.
- Generate customized home, jewelry, and party plans on demand to help users manage their finances effectively.
- Summarize long planning requirements or financial notes into concise, easy-to-read guides.
- Recommend structured, stage-wise planning roadmaps for any financial or lifestyle goal the user specifies.

4. Features

Home Planner & Recommendations

Accepts budget details or household goals and returns a structured plan covering expense allocation, savings strategies, and smart tips.

Jewelry Planner & Recommendations

Helps users plan their jewelry purchases, estimate gold/diamond weights, manage making charges, and structure savings milestones over time.

Party Planner & Recommendations

Organizes social gatherings by calculating guest estimates, venue costs, catering budgets, and checklist schedules.

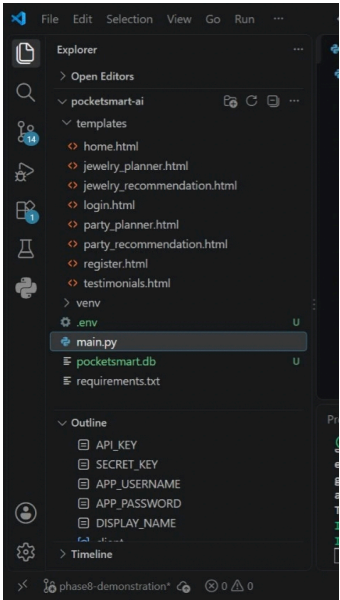
Smart Dashboard & History

Maintains user sessions, interactive inputs, and easy navigation across different planning modules.

5. Modules

PocketSmart AI's backend is organized into small, single-purpose Python modules, each responsible for one feature. This keeps the AI-prompting logic for every task isolated and easy to maintain.

File / Folder	Description
main.py	FastAPI application; defines all REST endpoints and routes each request to the correct module.
templates/	Jinja2 HTML templates for the frontend (base.html, dashboard.html, login.html, register.html, home_planner.html, jewelry_planner.html, party_planner.html, etc.).
static/	Styling and scripts for the frontend page (style.css, main.js).
tests/	Automated test cases for the backend modules.
.env	Stores the Gemini API key and other configuration values.



6. Module Description

main.py

Wires everything together as a FastAPI application: it defines all REST endpoints (/, /login, /dashboard, /home-planner, /jewelry-planner, /party-planner), manages user sessions securely, processes form inputs, handles Gemini API retries, and renders the Jinja2

HTML templates.

templates/

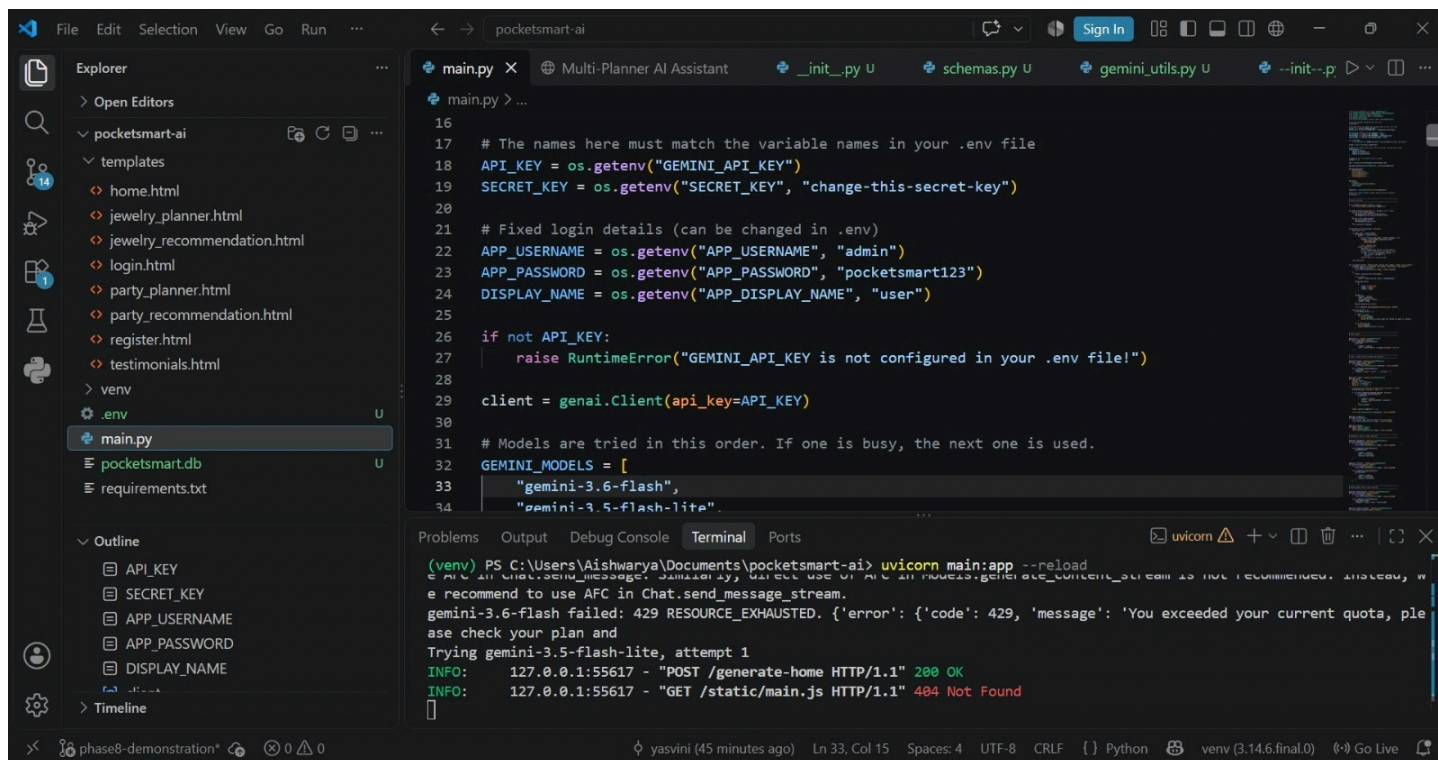
Contains all the Jinja2 HTML frontend views (base.html, dashboard.html, login.html, home_planner.html, jewelry_planner.html, party_planner.html, recommendation files, etc.) responsible for rendering the user interface dynamically.

static/

Stores styling assets and frontend scripts (style.css) that dictate the minimalist and responsive layout of PocketSmart AI.

.env

Secure configuration file that securely holds critical credentials including the Gemini API key, secret keys, and fixed login parameters without exposing them in version control.



The screenshot shows a VS Code editor with the file explorer on the left displaying the project structure. The main editor shows the `main.py` file with the following code:

```
16
17 # The names here must match the variable names in your .env file
18 API_KEY = os.getenv("GEMINI_API_KEY")
19 SECRET_KEY = os.getenv("SECRET_KEY", "change-this-secret-key")
20
21 # Fixed login details (can be changed in .env)
22 APP_USERNAME = os.getenv("APP_USERNAME", "admin")
23 APP_PASSWORD = os.getenv("APP_PASSWORD", "pocketsmart123")
24 DISPLAY_NAME = os.getenv("APP_DISPLAY_NAME", "user")
25
26 if not API_KEY:
27     raise RuntimeError("GEMINI_API_KEY is not configured in your .env file!")
28
29 client = genai.Client(api_key=API_KEY)
30
31 # Models are tried in this order. If one is busy, the next one is used.
32 GEMINI_MODELS = [
33     "gemini-3.6-flash",
34     "gemini-3.5-flash-lite".
```

The terminal window at the bottom shows the following output:

```
(venv) PS C:\Users\Aishwarya\Documents\pocketsmart-ai> uvicorn main:app --reload
INFO: 127.0.0.1:55617 - "POST /generate-home HTTP/1.1" 200 OK
INFO: 127.0.0.1:55617 - "GET /static/main.js HTTP/1.1" 404 Not Found
```

7. System Workflow

The application follows a simple request/response flow between the browser, the FastAPI backend, and the Gemini API:

- User opens PocketSmart AI in the browser (<http://127.0.0.1:8000>).
- User authenticates via Login/Register pages and navigates to the Dashboard.
- User selects a planning task from the dropdown (Home Planner, Jewelry Planner, Party Planner, etc.) and enters details.
- Frontend sends the input to the FastAPI backend as a request.
- `main.py` routes the request to the matching planning logic.
- The module builds a prompt and calls `gemini_client.py`, which sends it to the Google Gemini API.
- Gemini generates a response, which the module formats and returns as JSON.
- Frontend displays the result in the Result panel below the input box without reloading the page.

8. Technologies Used

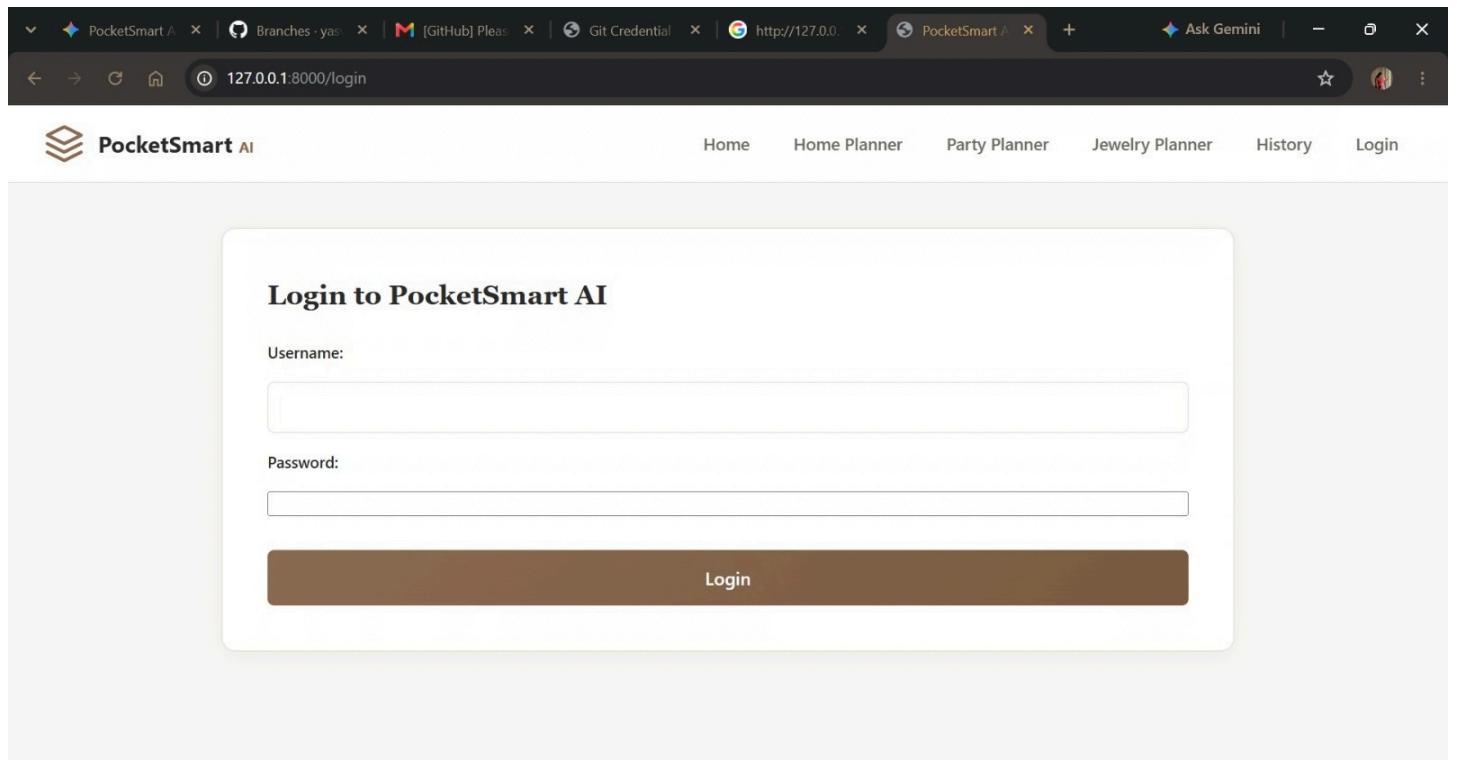
Technology	Role
Python 3.10+	Core programming language for the backend.
FastAPI	Backend web framework used to define REST endpoints and handle requests.
Uvicorn	ASGI server used to run the FastAPI application with live reloading.
Jinja2	HTML templating engine used to render dynamic templates and link static assets.
Google Gemini API	Generative AI model that powers smart planning, budgeting, and recommendations.
HTML5 & CSS3	Frontend page structure and responsive styling.
.env configuration	Used to store API keys and environment settings securely.
VS Code & Git	Primary development environment and version control system.

9. UI / Screen Description

The application opens with secure authentication interfaces (login.html and register.html) leading into a centralized dashboard (dashboard.html) designed for seamless multi-planner navigation.

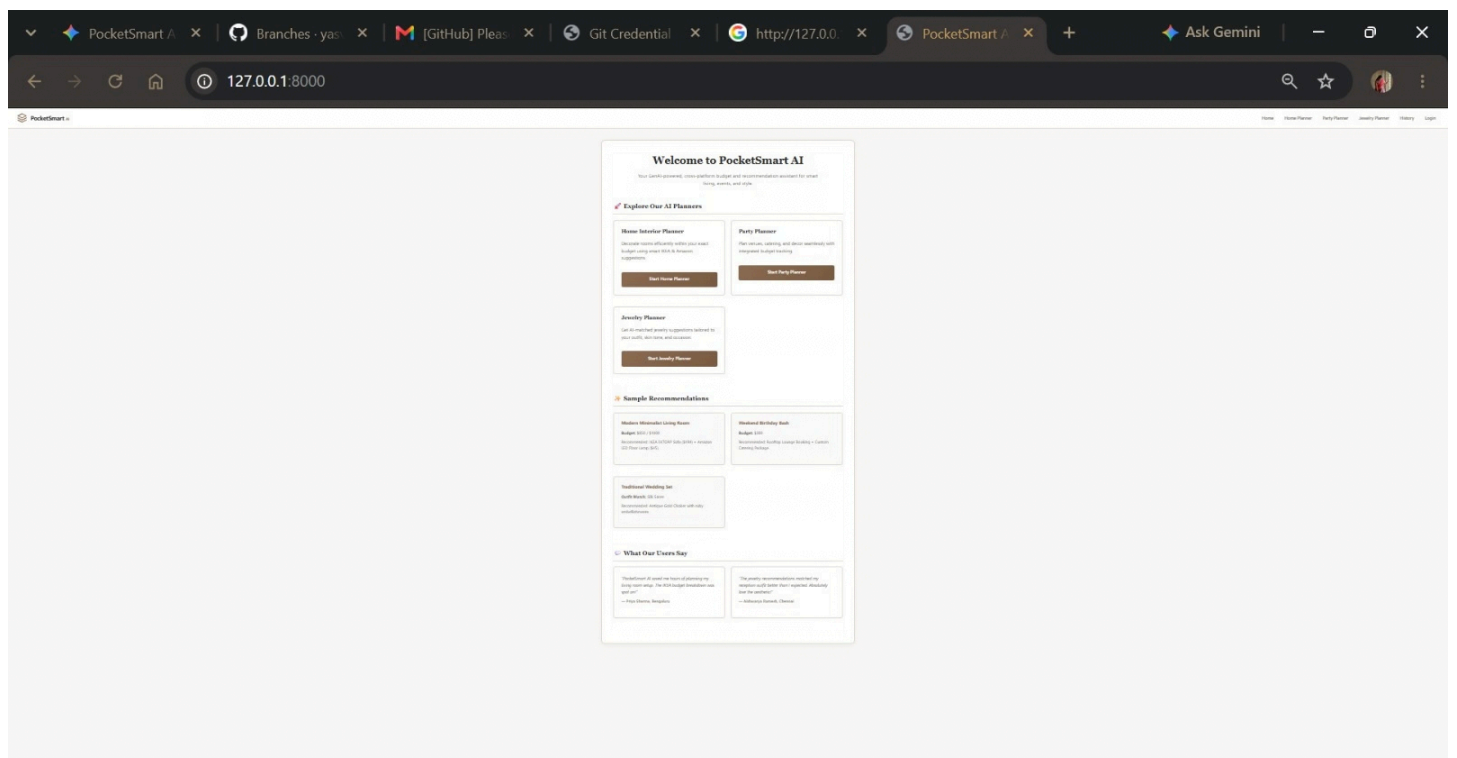
a. Login & Register Screens

The user authentication interface allows users to securely access their accounts. Upon successful verification, they are redirected to the main dashboard where all planning modules are accessible.



b. Dashboard

Once the user logs in or registers, a dashboard appears featuring the three AI planners, sample recommendations, and a dedicated “What Our Users Say” space that showcases user reviews.



c. Home Planner

Description: A clean, centralized form card where users can input their financial and planning requirements.

Input Fields:

- Total Budget (\$): Enter the maximum target amount (e.g., \$1500).

- Room Types: Specify rooms separated by commas (e.g., Living Room, Bedroom).
- Specific Items & Requirements: Detailed text area to specify item preferences such as lighting, ceiling fans, or sofa sets.

Action: Clicking the “Generate AI Recommendations” button securely sends the payload to the FastAPI backend.

PocketSmart AI

HomeHome PlannerParty PlannerJewelry PlannerHistoryLogin

Home Interior Budget Planner[cite: 1]

Total Budget (\$):

1500

Room Types (comma separated):

bedroom

Specific Items & Requirements:

lights

Generate AI Recommendations

d. AI-Powered Recommendation Output (/generate-home)

Interface Description: Displays the structured result generated dynamically by the Google Gemini API.

Features:

- Renders comprehensive interior design concepts, furniture breakdowns, and cost allocations based on the user's budget.
- Includes a “Back to Planner” navigation button to easily modify inputs or switch modules.

PocketSmart AI

HomeHome PlannerParty PlannerJewelry PlannerHistoryLogin

Home Interior AI Recommendations

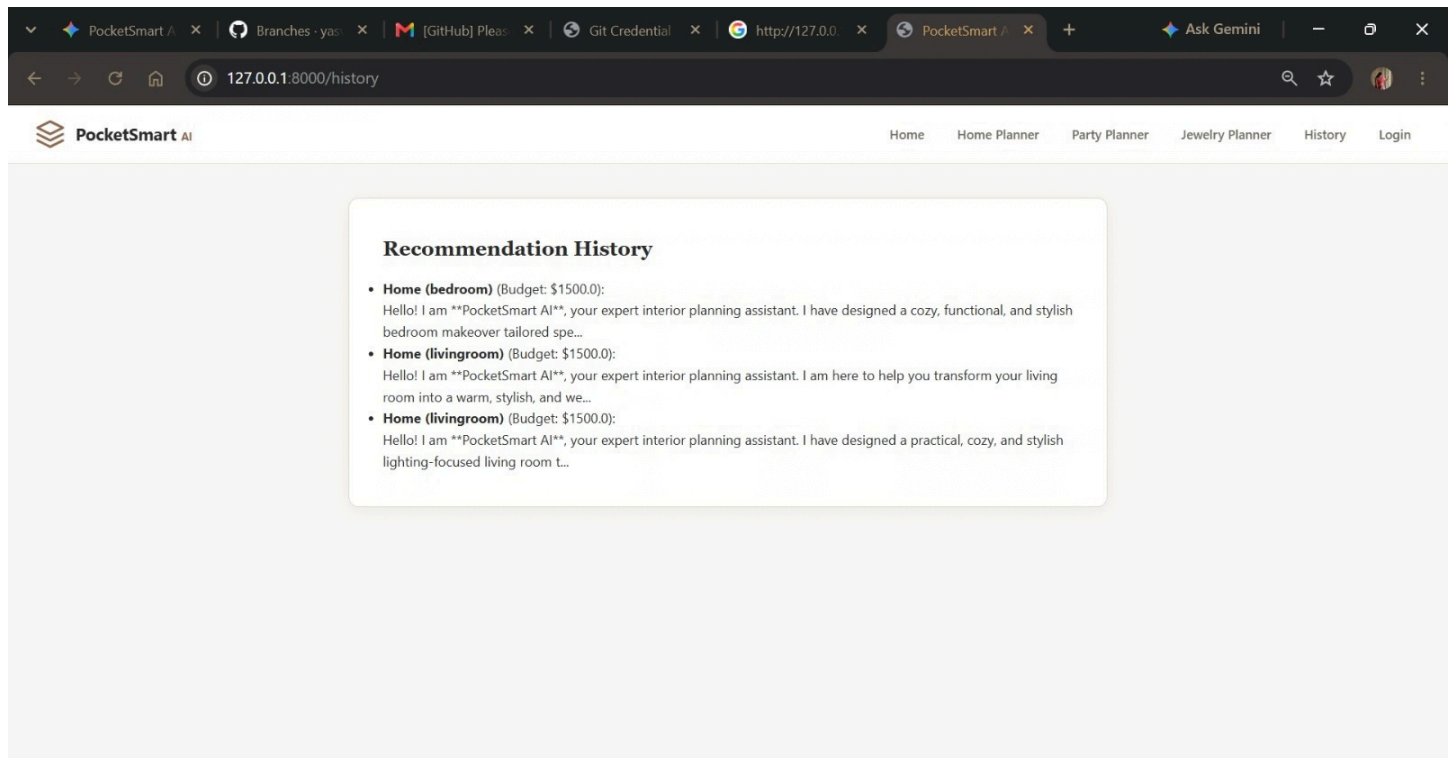
Back to Planner

Hello! I am **PocketSmart AI**, your expert interior planning assistant. I have designed a cozy, functional, and stylish bedroom makeover tailored specifically to your \$1,500 budget, with a special focus on upgrading your lighting. Here is your custom interior plan: ---
1. Interior Concept: "Warm Modern Minimalist" This design focuses on creating a calm, restful sanctuary using a neutral color palette (warm whites, soft grays, and natural wood tones). Because lighting is your main requirement, the concept uses **"layered lighting"**—combining ambient, task, and accent lighting—to make the bedroom feel larger, cozier, and highly functional for both relaxing and reading. ---
2. Recommended Furniture & Items
Bed Frame: Simple upholstered platform queen bed in charcoal gray.
Mattress: Medium-firm memory foam mattress (boxed/bed-in-a-box).
Nightstands: Set of 2 minimalist wooden nightstands with open shelving.
Main Ceiling Light: Dimmable modern semi-flush mount LED ceiling light (warm-to-cool adjustable).
Task/Bedside Lighting: Set of 2 sleek swing-arm wall sconces (plug-in style, so no hardwiring required).
Accent Lighting: 16-ft warm white LED strip light for behind the headboard (creates a floating, ambient glow).
Textiles: Neutral-toned duvet cover set, 4 blackout curtains, and a soft area rug (5x7 ft). ---
3. Estimated Cost for Each Item
Upholstered Platform Bed Frame: \$220.00
Memory Foam Mattress: \$350.00
Set of 2 Nightstands: \$110.00
Dimmable LED Ceiling Light: \$75.00
Set of 2 Plug-in

e. Recommendation History Tracking (/history)

Interface Description: Maintains an in-memory chronological record of all past AI generation sessions.

Details Displayed: Lists previous planning categories (e.g., Home (bedroom), Home (living room)), associated budgets, and preview snippets of generated insights.



f. Jewelry Planner

Description: A dedicated interface designed to help users plan gold and diamond purchases systematically. Users type in their target weight, current gold rates, or monthly saving capacity.

AI Processing & Response: The system processes the parameters via Gemini to calculate making charges, estimate target timelines, and provide structured installment savings schedules.

g. Party Planner

Description: A specialized event management module where users provide details like guest count, event type, and budget limits.

AI Processing & Response: Gemini evaluates the event requirements to generate a comprehensive cost estimation, venue checklist, and catering budget breakdown directly on the result panel without page reloading.

10. Project Implementation

10.1 Setting Up and Running the Application

The backend is started locally using Uvicorn from a Python virtual environment with the command:

```
.venv\Scripts\python.exe -m uvicorn main:app --reload
```

Once the server is running, the application is accessed locally by navigating to `http://127.0.0.1:8000` in a web browser.

10.2 Functional Testing

The application was tested by exercising each planning module end-to-end through the browser:

- Verifying secure authentication flows (Login/Register).
- Creating home budgets and checking structured financial breakdown responses.
- Simulating jewelry savings schedules and purchase estimations.
- Generating comprehensive party cost analyses and checklists.

11. Future Enhancements

- User Accounts & Financial Tracking — securely saving user budgets and plans across sessions.
- Export Options — letting users download generated plans and budget summaries as PDF or CSV files.
- Mobile-Friendly Interface — adapting the layout for responsive mobile and tablet usage.
- Analytics Dashboard — visual charts showing budget allocation and savings progress over time.

12. Conclusion

PocketSmart AI brings multiple planning tasks — home budgeting, jewelry purchasing, party organization, and intelligent recommendations — into one lightweight FastAPI application powered by the Google Gemini API. By routing requests through a structured interface and modular backend components, the project delivers a powerful, user-friendly AI assistant with a minimal infrastructure footprint.